

Práctica 4

Martínez Mejía Eduardo
Victoria Morales Ricardo Maximiliano
Torres Nava Hazel

October 9, 2026

Enlace a Repositorio

En el siguiente repositorio, se podrán ver los scripts que ejecutamos:

<https://github.com/Concurrente-2027-1/concurrency-masters>

1 Ejercicio

Computadora de Hazel

8 hilos

Table 1: Comparación de tiempos de ejecución de los SpinLocks y Counter

SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant	Counter Atomic
400 tareas, 4 hilos	7.239929 ms	6.874697 ms	22.274338 ms	7.541869 ms	7.561521 ms	8.260778 ms	7.595321 ms	9.527824 ms
1000 tareas, 4 hilos	8.581935 ms	9.019999 ms	44.500160 ms	9.881929 ms	9.849642 ms	10.783978 ms	9.284249 ms	14.546558 ms
1000 tareas, max-1 hilos	12.367856 ms	11.091532 ms	39.194550 ms	10.184872 ms	12.441372 ms	16.886750 ms	10.147002 ms	17.395502 ms

Table 2: Tiempos de ejecución de los SpinLocks con 100 tareas

SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant
100 tareas, 4 hilos	7.898775 ms	7.639880 ms	38.214012 ms	10.038253 ms	8.140094 ms	7.979318 ms	7.838378 ms
100 tareas, max-1 hilos	7.269787 ms	7.820631 ms	29.067777 ms	8.361605 ms	7.946841 ms	8.757848 ms	8.711055 ms

Computadora de Ricardo

8 hilos

Table 3: Primera tabla - tarea contador

SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant	Counter Atomic
400 tareas, 4 hilos	12.640708ms	12.472380ms	39.163302ms	14.015694ms	13.809928ms	14.570623ms	13.684095ms	13.506649ms
1000 tareas, 4 hilos	16.19856ms	18.02366ms	39.339951ms	16.75176ms	17.383544ms	17.602821ms	17.180246ms	16.462668ms
1000 tareas, max-1 hilos	17.01906ms	18.499139ms	41.755689ms	73.053749ms	29.639953ms	18.632108ms	18.73319ms	17.464599ms

Table 4: Segunda tabla - tarea matriz

SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant
100 tareas, 4 hilos	101.332777ms	89.059255ms	104.78257ms	92.286582ms	94.976472ms	94.5381319ms	98.276989ms
100 tareas, max-1 hilos	96.388506ms	89.701945ms	90.958184ms	90.432276ms	98.619317ms	97.731839ms	99.965865ms

Computadora de Eduardo

20 hilos

Table 5: Comparación de tiempos de ejecución de los SpinLocks y Counter

SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant	Counter Atomic
400 tareas, 4 hilos	5.8517 ms	5.7631999 ms	37.4405 ms	7.1457999 ms	6.2629 ms	6.5710999 ms	6.1106 ms	6.1949999 ms
1000 tareas, 4 hilos	7.7317ms ms	8.2132m ms	31.9162 ms	8.4302999 ms	8.4219999 ms	9.1561m ms	7.9893 ms	6.9697999 ms
1000 tareas, max-1 hilos	8.2755999 ms	7.9644999 ms	36.7996 ms	15.9125 ms	9.1744 ms	8.7466999 ms	8.2923 ms	7.8198 ms

Table 6: Tiempos de ejecución de los SpinLocks con Matrices compartidas

SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant
100 tareas, 4 hilos	8.6824 ms	6.9913 ms	29.758899 ms	8.1477 ms	8.4271 ms	7.824 ms	7.2024 ms
100 tareas, max-1 hilos	8.1169 ms	7.6044 ms	63.714 ms	8.1572 ms	8.0392 ms	7.923 ms	7.9446999 ms

2 Preguntas

3. Escojan al integrante del equipo con mayor número de hilos. ¿Cuáles son las 2 implementaciones más rápidas del inciso 1? ¿A qué crees que se debe?

TTAS y **CounterAtomic** fueron las más rápidas con todos los hilos.

TTAS reduce la contención al esperar leyendo el estado antes de intentar tomar el lock y solo intenta cuando lo ve libre. Por otro lado **CounterAtomic** puede ser aún más rápido porque incrementa con una operación atómica y evitando totalmente el costo de ejecución de tomar y dejar un lock.

Sin embargo podemos ver que **TAS**, **ALock**, **CLHLock**, **Reentrant** se acercan mucho a sus tiempos de ejecución registrados, así que debe considerarse también el estado de cada procesador cuando se pidió ejecutarse, por lo que una sola medición no garantiza que sean mejores. Deben hacerse más mediciones.

4. Escojan al integrante con mayor número de hilos. ¿Cuáles son las 2 implementaciones más rápidas del inciso 2? ¿A qué crees que se debe?

TTAS y **CLHLock** son las más rápidas con todos los hilos.

TTAS puede funcionar bien con la carga de las matrices al evitar que todos quieran hacer escrituras atómicas constantemente en la matriz, y **CLH** tiene la ventaja de que los hilos esperan en una cola, reduciendo la competencia de muchos hilos por una ubicación compartida mientras la sección crítica dura más.

5. ¿Cuáles son las implementaciones que satisfacen la propiedad de Justicia? (Por el momento ignora **ReentrantLock**). Argumenta cómo lo hacen.

Las implementaciones que satisfacen la propiedad de **Justicia** son **ALock**, **CLHLock** y **MCSLock**, ya que organizan la entrada a la sección crítica siguiendo un orden FIFO (First-In, First-Out).

- **ALock:** Utiliza un arreglo circular y un contador atómico global mediante `getAndIncrement()`. Cada hilo obtiene un índice al llegar y espera su turno. Como los turnos avanzan de forma secuencial, los hilos entran a la sección crítica en el orden en que solicitaron el candado.
- **CLHLock:** Utiliza una cola implícita de nodos y actualiza atómicamente la variable `tail` mediante `getAndSet()`. Cada hilo se coloca al final de la cola y espera a que su predecesor libere el candado. De esta manera, los hilos respetan el orden de llegada y ninguno puede adelantarse a los que están esperando antes.

- **MCSLock:** Utiliza una lista ligada explícita en la que cada hilo agrega su nodo al final de la cola mediante `getAndSet()`. Cada hilo espera revisando su propia variable `locked`. Cuando se libera el candado, el hilo actual despierta a su sucesor, respetando el orden FIFO y evitando que algún hilo espere indefinidamente.

Por otro lado, `TASLock`, `TTASLock` y `BackoffLock` no garantizan la propiedad de justicia, porque los hilos compiten directamente por adquirir el candado cuando se libera. Esto permite que un hilo consiga entrar varias veces antes que otro, provocando que este último pueda esperar indefinidamente.